C 语言示例教材

本材料由 AI 原创编写，用于 SolveMate 的教材阅读与文字询问示例，不对应指定学校或教材。示例采用标准 C11，覆盖 6 个主题，配套题号 C001–C012。

每章包含概念说明、完整程序、输出与易错点。示例中的程序可以独立编译运行。

1 程序结构与类型

一个简单的 C 程序通常由头文件、函数定义和语句组成。main 是程序入口，return 0 表示正常结束。变量声明说明名称和类型；类型决定可表示的数据以及运算规则。

printf 根据格式字符串输出参数。%d 对应 int，%u 对应 unsigned int，%f 对应经参数传递的 double。字符串中的 \n 表示换行。整数除法与浮点除法要根据运算发生时的操作数类型区分。

C001 表达式与输出

```
#include <stdio.h>
int main(void) {
    int a = 2, b = 5;
    printf("%d\n", a + b);
    return 0;
}
```

输出：

```
7
```

a 和 b 都是 int，a + b 得到整数 7；%d 用于输出 int，\n 表示换行。

易错点：格式占位符需要与参数类型匹配。

C002 整数除法与浮点除法

```
#include <stdio.h>
int main(void) {
    printf("%d %.1f\n", 5 / 2, 5.0 / 2);
    return 0;
}
```

输出：

```
2 2.5
```

5 / 2 的两个操作数都是整数，结果为 2；5.0 / 2 中的 5.0 是 double，结果为 2.5。

易错点：运算完成后再赋给浮点变量，不能恢复整数除法已经丢掉的小数部分。

2 分支与循环

分支根据条件选择执行路径；循环重复执行一段语句。if 的条件非零时进入真分支，为零时进入假分支。不要把比较运算 == 和赋值运算 = 混淆。

for 循环依次执行初始化、条件检查、循环体与更新。分析循环可以列出每次迭代前后的变量值。取模运算 % 用于整数；对本例的正整数，n % 2 == 0 可以判断偶数。

C003 循环累加

```
#include <stdio.h>
int main(void) {
    int sum = 0;
    for (int i = 1; i <= 5; ++i) {
        sum += i;
    }
    printf("%d\n", sum);
    return 0;
}
```

输出：

```
15
```

i 依次取 1、2、3、4、5，sum 依次变成 1、3、6、10、15。

易错点：循环结束条件 i <= 5 包含 5，不能误读为只累加到 4。

C004 条件选择与取模

```
#include <stdio.h>
int main(void) {
    int n = 8;
    if (n % 2 == 0) {
        puts("even");
    } else {
        puts("odd");
    }
    return 0;
}
```

输出：

```
even
```

8 % 2 的余数为 0，因此条件为真，执行第一个分支；puts 会在字符串之后追加换行。

易错点：== 是比较，= 是赋值；二者在条件表达式中的作用不同。

3 函数

函数把输入、处理过程和返回值组织成可复用单元。C 的参数传递是按值传递：形参获得实参值的副本，修改形参本身不会改变调用者变量。

递归指函数直接或间接调用自身。分析递归需要确定终止条件、问题规模如何缩小、返回时如何合并结果。有限宽度的整数会限制结果范围，因此不能把小数值示例推广为任意输入都安全。

C005 函数参数按值传递

```
#include <stdio.h>
int add_ten(int value) {
    value += 10;
    return value;
}
int main(void) {
    int n = 3;
    int result = add_ten(n);
    printf("%d %d\n", n, result);
    return 0;
}
```

输出：

```
3 13
```

形参 value 是 n 的值的副本，修改 value 不改变 n；返回值为 13，赋给 result。

易错点：形参与实参名字可以不同；修改普通形参不会直接修改调用者的变量。

C006 递归与终止条件

```
#include <stdio.h>
unsigned int factorial(unsigned int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
int main(void) {
    printf("%u\n", factorial(5));
    return 0;
}
```

输出：

```
120
```

递归逐次调用 factorial(4)、factorial(3)、factorial(2)、factorial(1)，再返回并相乘得到 120。

易错点：递归必须有可到达的终止条件；固定宽度整数不能表示任意大的阶乘。

4 数组与字符串

数组是一组相同类型的连续元素，通过从 0 开始的下标访问。长度为 n 的数组有效下标为 0 到 n−1。带初始化器但未明确给值的剩余元素会按规则初始化为零。

C 字符串以空字符 '\0' 结束。strlen 统计结尾空字符之前的字符数；sizeof 对象给出对象所占字节数。在定义 char word[] = "cat" 的作用域内，sizeof word 为 4，strlen(word) 为 3。

C007 数组的部分初始化

```
#include <stdio.h>
int main(void) {
    int values[4] = {1, 2};
    printf("%d %d %d %d\n",
           values[0], values[1], values[2], values[3]);
    return 0;
}
```

输出：

```
1 2 0 0
```

初始化器只提供前两个元素，数组中其余元素按规则初始化为 0。有效下标为 0 到 3。

易错点：这不代表没有初始化器的自动存储期局部数组也会自动清零。

C008 字符串长度与存储大小

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char word[] = "cat";
    printf("%zu %zu\n", sizeof word, strlen(word));
    return 0;
}
```

输出：

```
4 3
```

word 数组存放 c、a、t 和结尾的空字符，共 4 个 char；strlen 返回空字符之前的字符数 3。

易错点：sizeof word 与 strlen(word) 不是同一个概念；对指针使用 sizeof 也不能得到所指字符串长度。

5 指针

指针保存对象的地址。&x 取得 x 的地址，*p 在 p 有效并指向对象时访问该对象。指针可以为空；空指针和无效指针都不能解引用。

数组表达式在许多使用场景中转换为首元素指针，sizeof 等场景是例外。指针运算以元素为单位，必须遵守同一数组对象范围的规则；可以形成尾后指针，但不能解引用它。传入变量地址，函数便可以通过指针修改该对象。

C009 指针与数组元素

```
#include <stdio.h>
int main(void) {
    int values[] = {10, 20, 30};
    int *p = values;
    printf("%d %d\n", *p, *(p + 2));
    return 0;
}
```

输出：

```
10 30
```

p 指向第一个 int 元素，*p 为 10；p + 2 指向第三个元素，解引用得到 30。

易错点：指针加 1 按所指类型移动一个元素，而不是固定移动一个字节。

C010 通过指针交换变量

```
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main(void) {
    int x = 3, y = 7;
    swap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```

输出：

```
7 3
```

调用时传入 x 和 y 的地址；swap 内的指针本身仍按值传递，但解引用操作访问并修改了原变量。

易错点：C 没有因为参数是指针就改成按引用传递；不要把指针值与所指对象混淆。

6 结构体与文件

结构体把多个成员组合成一个值。对象通过点号访问成员，结构体指针可以通过 -> 访问成员。赋值复制结构体成员的值；包含指针成员时，不会自动复制指针所指的动态数据。

标准文件接口用 FILE * 表示流。打开文件或创建临时文件后应检查结果。文件位置会随读写推进；rewind 把位置移回开头。fgets 最多读取指定容量减一的字符，并在成功时补上结尾空字符；读到换行时会保留换行。最后使用 fclose 关闭流。

C011 结构体的值复制

```
#include <stdio.h>
struct Student {
    int id;
    double score;
};
int main(void) {
    struct Student first = {7, 92.5};
    struct Student second = first;
    second.score = 80.0;
    printf("%.1f %.1f\n", first.score, second.score);
    return 0;
}
```

输出：

```
92.5 80.0
```

结构体赋值复制各成员的值。本题成员不含指针，修改 second.score 不会影响 first.score。

易错点：若结构体含指针成员，赋值复制的是指针值，不会自动深复制所指内存。

C012 文件位置与读取

```c
#include <stdio.h>
int main(void) {
    FILE *file = tmpfile();
    if (file == NULL) {
        return 1;
    }
    fputs("C language\n", file);
    rewind(file);
    char buffer[32];
    if (fgets(buffer, sizeof buffer, file) == NULL) {
        fclose(file);
        return 1;
    }
    printf("%s", buffer);
    fclose(file);
    return 0;
}
```

输出：

```
C language
```

写入后调用 rewind 将文件位置移回开头，再由 fgets 读取。fgets 保留读到的换行符，所以 printf 不需要额外添加换行。

易错点：读写文件要检查打开和读取结果；字符数组的容量不等于当前字符串的长度。